Introducción.

Las redes neuronales profundas funcionan bien con:

- Datos organizados jerárquicamente:
  - **Imágenes**. Píxeles que forman líneas, regiones, etc.
  - **Palabras**. Cada palabra está formada por fonemas.
  - **Texto**. Formados por caracteres, palabras, oraciones, etc.
- Son capaces de descubrir características útiles, aunque sin exponerlas a los humanos (cajas negras).
  - Se puede utilizar para ingeniería de características.
    - Selección de las características más importantes de las instancias.

Principales redes neuronales profundas:

- **Perceptrón multicapa**.
  - Redes completamente conectadas hacia adelante
  - Con al menos una capa oculta, además de la de entrada y salida.
- **Red neuronal convolucional**.
  - Redes conectadas hacia adelante
  - Contienen capas encargadas de aplicar un determinado filtro a los valores recibidos
  - Internamente se producen mapas de activación de n dimensiones.
  - Son parecidas a como se organizan las neuronas en la capa visual del cerebro.
- **Redes recurrentes**.
  - Tienen un estado interno para almacenar parte de la información que se le ha pasado a la red.
  - La salida es una combinación entre los nuevos datos y el estado interno.
  - Con cada iteración de nuevos datos, el estado interno se actualiza.
- **Autoencoders**.
  - La salida es de la misma dimensión que la entrada, permitiendo aprender mejores representaciones básicas.

En este tipo de redes se usa el algoritmo del gradiente descendente con momentum, $\mu\Delta w$:

$$\Delta w' = \mu\Delta w - \lambda\big(\nabla \mathrm{J}(\mathrm{w})\big)$$

$$w' = w + \Delta w'$$

El momentum ayuda al gradiente a alcanzar el mínimo en menos iteraciones.

Algunas aplicaciones del aprendizaje profundo:

- Reconocimiento de la carretera y situaciones de emergencias en la asistencia a la conducción de los coches.
- Visión artificial.
- Aplicaciones médicas en los campos de reconocimiento de imágenes (rayos X, resonancias, etc.) o en el análisis de historiales clínicos para detección de patrones de enfermedades.
- Traducción de texto entre idiomas.
- Conversaciones a tiempo real con humanos.

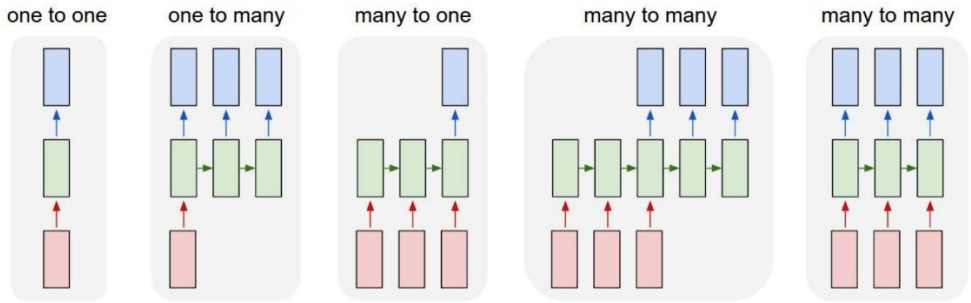
Redes Neuronales Recurrentes.

- Destinadas a secuencias de datos.
- Tienen un estado interno obtenido a partir de el estado anterior y la entrada.
    - $h_t = f(h_{t-1}, x_t)$
    - Normalmente se usa la tangente hiperbólica:
        - $h_t = tanh(W_{h-1} \cdot h_{t-1} + W_x \cdot x_t)$
- La salida se calcula a partir del estado calculado.
    - $y_t = softmax(W_h \cdot h_t)$
        - Softmax: si se quiere obtener un conjunto de probabilidades que sume 1
        - Sigmoide: si las probabilidades calculadas son independientes entre ellas.
- Durante el entrenamiento de **una secuencia temporal** (ya sea una instancia o varias), los pesos NO VARÍAN. Solo varía cuando se tiene la salida definitiva, al finalizar el procesamiento de toda la secuencia.
- **Diferentes arquitecturas según el objetivo de la red**.

  Supongamos que x_i son instancias formadas por k características e y_i es una salida de tamaño l (clases, valor concreto o secuencia de valores).

    - **Una a una**.
        - Cuando se quiere obtener una única salida y_i para una secuencia con una única instancia x_i.
        - También denominado **secuencia a secuencia** (aunque no se proporcione una secuencia y no se reciba una secuencia temporal).
        - Por ejemplo: POS tagging.

- **Una a muchas**.
  - Una única instancia de entrada x_i genera una secuencia (o también llamado **vector**) de salidas $y_i, \cdots, y_{i+n}$. Los valores $n-1$ de la secuencia de entrada serán 0 o el mismo valor x_i.
  - Denominado **secuencia a vector**.
  - Por ejemplo: imagen a texto.
- **Muchas a una**.
  - De una secuencia de instancias (vector) $x_i, \cdots, x_{i+n}$ se tiene en cuenta únicamente la última salida y_{i+n}.
  - En keras se debe indicar `return_sequence` a `False`.
  - Denominado **vector a secuencia**.
  - Por ejemplo: generación de imagen a partir de texto.
- **Muchas a muchas**.
  - Tiene dos casos particulares:
    - De una secuencia de instancias (vector) $x_i, \cdots, x_{i+n}$, se tienen en cuenta las salidas $y_{i+n}, \cdots, y_{i+n+m-1}$.
    - De una secuencia de instancias (vector) $x_i, \cdots, x_{i+n}$, se tienen en cuenta las salidas $y_i, \cdots, y_{i+n}$. Es decir, las predicciones hechas para cada instancia temporal.



- Formas de generar predicciones de una secuencia. Si se quiere predecir m valores se puede:
  - Dada una secuencia de entrada $x_i, \cdots, x_{i+n}$, se genera la predicción y_{i+n} y se añade a la secuencia generando la entrada $x_{i+1}, \cdots, x_{i+n}, y_{i+n}$ para predecir y_{i+n+1}.
  - Se generan las m predicciones a la vez añadiendo una capa densa al final con m salidas.

```
model = keras.models.Sequential([
    keras.layers.SimpleRNN(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.SimpleRNN(20),
    keras.layers.Dense(10)
])
```

  - Se devuelve las predicciones realizadas en cada instancia temporal de la secuencia de entrada (segundo caso muchas a muchas).

```
model = keras.models.Sequential([
    keras.layers.SimpleRNN(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.SimpleRNN(20, return_sequences=True),
    keras.layers.TimeDistributed(keras.layers.Dense(10))
])
```

- Dos problemas con las redes recurrentes:
  - **Desvanecimiento del gradiente**. Datos pasados dejan de tener influencia en datos recientes.
  - **Explosión del gradiente**. Gradiente tan grande que llega a infinito.
- Dos principales modelos de células recurrentes:
  - **LSTM**.
    - Capaces de recodar a largo plazo.
    - Dos estados:
      - **Estado de la célula**, c. Memoria a largo plazo.
      - **Estado oculto**, h. Memoria a corto plazo y salida.
    - La influencia de los estados se determina con puertas:
      - **Puerta de entrada**, ***input gate***. Determina qué nueva información se va a almacenar en el estado c.
      - **Puerta de salida, *output gate*.** Determina la salida de la célula (estado h) en base del nuevo estado c y del estado anterior h y la entrada.
      - **Puerta del olvido, *forget gate*.** Determina cuando del estado anterior c debe mantenerse para el momento actual.
    - Resistentes al problema del desvanecimiento del gradiente.
  - **GRU**.
    - Variante simplificada de la LSTM.
      - Requiere menos recursos que la LSTM (menos parámetros).
      - No desempeña igual de bien y tiende al sobreajuste.
    - Solo tiene un estado, el oculto.
    - Dos puertas:
      - **Puerta de reinicio**. Determina cuanta información del estado anterior va a eliminarse.
      - **Puerta de actualización**. Determina cuanta información de la entrada va a añadirse al nuevo estado.
- Diferentes configuraciones de este tipo de redes:
  - **Redes bidireccionales**.
    - La entrada se procesa tanto de izquierda a derecha como de derecha a izquierda.
    - La salida es una combinación de ambos procesos.

    - **Redes stateful**.
        - El estado oculto de mantiene entre entrenamientos de diferentes conjuntos de datos.

Hopfield Networks

- Grafos completamente conectados.
- Debe haber una neurona por píxel de la imagen.
- El entrenamiento de las neuronas se basa en la regla de Hebb: el peso entre dos neuronas se incrementa si ambos pixeles están "encendidos", y decrementa si no.
- Lleva asociada una función de energía que decrementa en cada iteración al realzar los cálculos de los pesos.
    - Puede darse el caso de que en estados de baja energía se tengan patrones que no aparecen en los datos. Estos patrones se llaman **patrones espurios**.
- A partir de los datos entrenados aprende un patrón.
- A partir de estos patrones, cuando se obtiene un nuevo dato se realiza una comparación con cada uno para detectar cuál es el más parecido.
- Consume mucha memoria en relación con el número de patrones que es capaz de aprender.
- Útil para detección de caracteres.

Máquinas de Boltzmann

- Grafos completamente conectados basados en neuronas estocásticas:
    - Cada neurona tiene una probabilidad de devolver 1 o 0.
    - La función de probabilidad se basa en la distribución de Boltzmann.
    - Dos grupos de neuronas:
        - **Visibles**. Las que reciben las entradas y devuelven las salidas.
        - **Ocultas**. Solo realizan cálculos.
- Nunca llegan a estabilizarse.
    - Se dice que se llega a un **equilibrio térmico** cuando se alcanza un estado en el que la configuración inicial se ha "olvidado" (no tiene nada que ver una con la otra).
- Es capaz de simular muchas distribuciones de probabilidad.
    - Modelo generativo.
- Útil en el caso de imágenes, para la reducción de ruido o completado, o clasificación.

Máquinas de Boltzmann Restringidas (RBM)

- Modelo no supervisado.
- Las neuronas de un mismo grupo no están conectadas entre sí.
- Se entrenan usando la **divergencia contrastiva:**
    - Dada una instancia de entrenamiento x, se establece el estado de las neuronas visibles.
    - Se calcula el estado de las neuronas ocultas, h.
    - Se calcula el estado de las neuronas visibles, x'.
    - Se vuelve a calcular el estado de las neuronas ocultas, h'.
    - Se ajustan los pesos usando estos cuatro estados y la ratio de aprendizaje.

Redes de creencias profundas (DBN)

- Consisten en RBM apiladas.
- También es un modelo no supervisado, aunque se puede usar entradas y salidas en la última capa con las etiquetas para comprobar si la especificación se está realizando correctamente.
    - Esto permite generar nuevas instancias activando una de estas células destinadas a las etiquetas.
- Calculan con mayor precisión las probabilidades al tener más niveles:
    - Las células de menor nivel aprenden características de bajo nivel.
    - Las de mayor nivel características más específicas.
- Ejemplos de utilización:
    - Generación de etiquetas a partir de imágenes y viceversa, uniendo dos RBM que hayan aprendido sobre cada uno por separado.

Mapas auto-organizativos.

- Algoritmo no supervisado.
- Se usan para generar una representación de baja dimensión de un conjunto de alta dimensión.
- Las neuronas se organizan generalmente en un plano 2D (para visualización).
- Cada neurona del plano está conectada con todas las neuronas de entrada.
- Las instancias de entrada activarán únicamente una neurona en el mapa.
    - Instancias muy parecidas activaran neuronas cercanas.
- El algoritmo consiste en hacer competir todas las neuronas entre sí:
    - Los pesos son inicializados aleatoriamente.

- Las neuronas calculan la distancia entre dichos pesos y la instancia de entrada.
- La que tenga el peso más cercano gana y modifica los pesos para que sean aún más cercanos.
    - Las neuronas vecinas también modifican en menos medida sus pesos para hacer que se parezcan.
- El proceso se repite para cada instancia de entrenamiento.

Autocodificadores

- Se basan en "copiar" la entrada en la salida lo más fiel posible.
- Son buenos para la reducción y detección de características.
    - Si solo se usan funciones de activación lineales (no hay función realmente) y la función de coste es el MSE, su comportamiento es el de PCA.
        - **La función de coste casi siempre será el MSE**.
- Son modelos generativos.
- Redes capaces de aprender representaciones densas de los datos, denominadas **representaciones latentes** o codificaciones.
- Arquitectura básica:
    - Compuestos por dos partes:
        - El **codificador.**
            - Convierte la entrada en la representación latente.
            - Capa de entrada de los datos (generalmente que los aplana) y una capa de salida con tantas como se quiera.
            - Si se usan concatenando una capa softmax al final puede usarse para clasificar los datos.
        - El **decodificador**
            - Transformar la representación latente en la salida.
            - Mismo número de entradas que la representación latente y una capa de salida con tantas como se tenía originalmente.
    - Puede haber **capas ocultas** entre ambos para mejorar la representación latente.
        - En el caso de autocodificadores destinados a imágenes, las capas ocultas pueden ser convolucionales y pooling.
    - Suele tener una arquitectura simétrica.
    - Se suele establecer los pesos de la parte decodificadora en relación con los pesos de la parte codificadora.

        - Generalmente se usa la transpuesta.
- Tipos de autocodificadores:
    - **Autocodificadores de reducción de ruido**.
        - Se añade una capa “dropuot” o de ruido gaussiano para rellenar el ruido.
    - **Autocodificador disperso**.
        - Obligan a que muchas de las neuronas están inactivas para una instancia de entrada.
        - Obliga a destinar las neuronas a una característica en concreto.
        - Aplica una penalización sobre la codificación generada si se aleja mucho de 0.
        - Dos tipos de penalización:
            - Regularización L1.
            - Divergencia Kullback-Leibler.
    - **Autocodificador variacional (VAE)**.
        - Las salidas proporcionadas son determinadas bajo probabilidad.
        - Son modelos generativos.
        - La codificación es la media y varianza de la instancia.
            - La codificación puede ser de más de una dimensión, habiendo una media y varianza por cada una.
            - Da igual la distribución de donde provenga, el codificador la toma como si fuera normal, generando un espacio latente.
        - La decodificación consiste en añadir ruido gaussiano a la codificación para obtener una salida ligeramente diferente.
        - La función de coste está formada por dos partes:
            - El MSE
            - La **pérdida latente**, que se expresa en función de la codificación de la media y la varianza de la codificación.

Redes generativas adversariales (GAN)

- Redes destinadas a la generación de nuevos datos a partir de los datos de entrenamiento.
- Están formadas por
    - **Generador.** Genera los nuevos datos a partir de una distribución aleatoria.

    - **Discriminador**. Intenta distinguir los datos reales de los falsos.
- El entrenamiento consiste en intentan conseguir que el generador consiga los datos más reales posibles mientras que el discriminador sea capaz de distinguir los falsos de los reales.
    - Primero se entrena el discriminador para que sea capaz de diferenciar entre falsas y reales.
    - Después se entrena el generador usando el discriminador, solo que todas las imágenes se etiquetan como reales para que el generador haga imágenes que el discriminador se trague.
- Una GAN es perfecto cuando el generador solo produce imágenes consideradas reales y el discriminador tiene un ½ de posibilidades de acertar. Este estado no se garantiza.
    - Se puede producir sesgo cuando el generador se centra en producir imágenes muy buenas de una sola clase y el discriminador se centra en diferenciar imágenes de una sola clase.
    - Se puede usar técnicas para evitar esto:
        - **Repetición de experiencias**.
        - **Discriminación de mini-lotes**.
- Se pueden introducir dentro del generador capas convolucionales.
    - Con la profundidad adecuada son capaces de generar imágenes muy realistas.
- Su uso abarca:
    - Super resolución.
    - Coloreado.
    - Edición de imágenes.
    - Transformación de dibujos en imágenes.
    - Predicción de frames.
    - Detección de debilidades en otros modelos.

Modelos de difusión

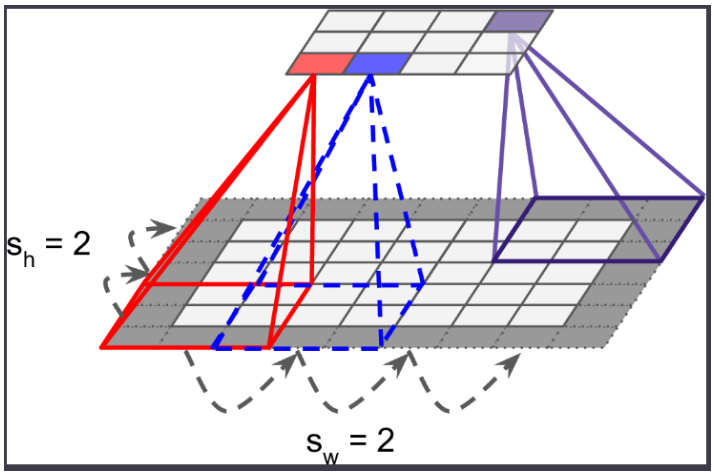
- Redes con el mismo propósito que las GAN.
- Generan mejores resultados y son más fáciles de entrenar.
- Tardan mucho más en generar salidas.
- Existe el modelo probabilístico de reducción de ruido por difusión, especializado en reducir el ruido de las imágenes.

Redes neuronales recursivas

- Redes diseñadas específicamente para procesar datos que tengan estructuras jerárquicas.
- En datos temporales son peores que las recurrentes.
- Descompone la entrada en hojas de un árbol, detectando las relaciones entre ellas y como forman la oración.
- Pueden recuperar tanto estructura granular como estructura jerárquica de alto nivel.
- Para el ajuste de los pesos durante el entrenamiento usa el algoritmo de retro propagación a través de estructura, que funciona desde la raíz del árbol hacia las hojas.
- Variaciones:
    - **Autoencoders Recursivos**. Intentan reconstruir la entrada. En procesamiento de lenguaje natural se intenta reconstruir el contexto.
    - **Redes de Tensores Neuronales Recursivos**.
- Usos:
    - Descomposición de imágenes.
    - Procesamiento de lenguaje natural.
    - Transcripciones de audio.

Redes neuronales convolucionales

- Cada capa de la red puede ser de 3 dimensiones:
    - Hay apiladas tantas matrices de 2 dimensiones con neuronas como filtros se desee tener en la capa.
    - Cada posición (i, j) de las matrices es una neurona.
- Compuestas principalmente por estos tipos de capas:
    - **Capa convolucional**.
        - Se denomina **campo receptivo o filtro** a la submatriz de neuronas que generarán una salida.
            - El resultado de aplicar un filtro es un **mapa de características**.
            - Puede existir solapamiento entre campos receptivos, determinando el tamaño de la matriz resultante.



- A mayor solapamiento, menor será el tamaño del mapa de características.
- Los filtros se definen automáticamente en el entrenamiento según sea el objetivo.
- Suelen tener salida ReLU.

▪ Una capa convolucional tiene tantos filtros como se quiera.

- La salida de la capa es la ponderación de los mapas de características.
- Dentro de una capa se puede indicar que se le añada un relleno de 0s a lo largo del borde de la entrada.
- Si se quiere rellenar los bordes: “SAME”.
- Si no se quiere rellenar los bordes: “VALID”.

○ **Capa de agrupación o pooling**.

▪ Sirven para reducir la carga computacional, encogiendo el resultado proveniente de otra capa.
▪ No tienen pesos.
▪ Debe definirse el método de reducción.
▪ El tamaño del filtro determinará el factor de reducción de la imagen.

- Un filtro de 2x2 hará resultar en una imagen de la mitad del tamaño.

▪ La capa de agrupación más común es **Max Pooling**.

- Un filtro de n x n SIN solapamiento.

○ **Capa de normalización**.

▪ Estabiliza el entrenamiento.
▪ Aplica una normalización a las entradas que reciba.

○ **Dropout**.

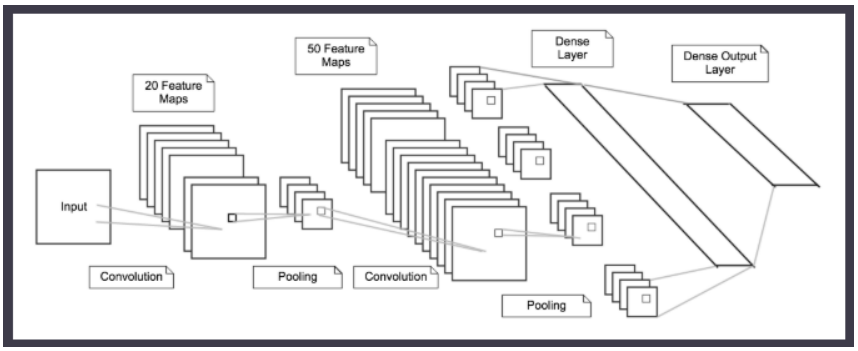
▪ Desactiva un porcentaje de neuronas de la capa para evitar el sobreajuste.

○ **Aplanado**.

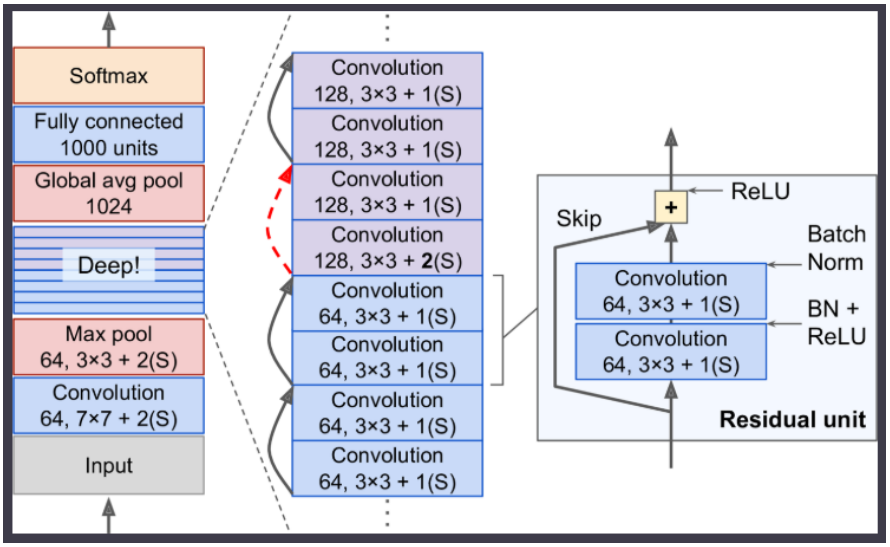
        - Transforma un mapa en un vector.
- Arquitecturas de las redes convolucionales:
    - **Básica**.

```
. Conv2D ( 64 ,
    input_shape
. MaxPooling2D (
. Conv2D ( 128 ,
. Conv2D ( 128 ,
. MaxPooling2D (
. Conv2D ( 256 ,
. Conv2D ( 256 ,
. MaxPooling2D (
. Flatten (),
. Dense ( 128 ,
. Dropout ( 0.5
. Dense ( 64 ,
. Dropout ( 0.5
. Dense ( 10 ,
```

        - Una o dos capas convolucionales (tantos filtros como se quiera) con capa de activación ReLU seguidas de una capa de normalización y otra de agrupación.
            - Idealmente incrementar el número de filtros de capa a capa: 32, 64, 128, etc...
        - La última capa convolucional le sigue una capa de aplanamiento.
        - La parte final de la red es una red neuronal completamente conectada.
        - Todas las capas tienen como salida ReLU, a excepción de la última que llevará la función de activación que corresponda según el objetivo:
            - Softmax si queremos la probabilidad exclusiva de pertenecer a cada una de las clases de salida, sumando entre todas 100%.
                - Generalmente para multiclase junto con categorical_crossentropy o sparse si son enteros.
            - Sigmoide si queremos la probabilidad de pertenecer a cada una de las clases de salida, con las probabilidades no teniendo que sumar 100%.
                - Generalmente para binaria junto con binary_crossentropy.
    - **LeNet-5**:
        - Tres capas convolucionales con dos de pooling entre ellas, todas con activación tanh.
        - A la última convolucional le sigue dos densamente conectadas, la primera con activación tanh y la última con RBF.
        - Esta red toma imágenes con el añadido de los bordes a 0, por lo que la red continuamente va reduciendo el tamaño.



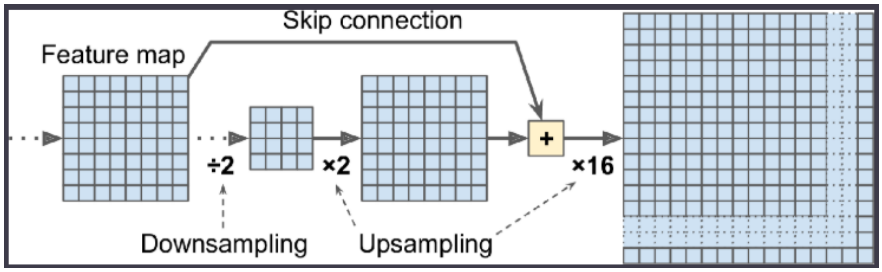
- **AlexNet**:
  - Es la estructura en la que se basa la arquitectura “básica”:
  - Al menos una capa convolucional seguida de una capa de máxima agrupación.
  - Capas de agrupamiento del tipo Max pooling con relleno “valid”.
  - Termina con 3 capas densamente conectadas: las dos primeras con ReLU y la última softmax.
- **ResNet-34**:
  - Formada por muchas capas con pocos parámetros cada una.
  - Utiliza el aprendizaje residual: a la salida producida por un conjunto de capas se le añade la entrada.
    - Esto permite acelerar el entrenamiento de la red.



  - Esta versión tiene 34 capas contando las capas convolucionales y la densamente conectada.
  - Keras dispone de ResNet-50 que espera imágenes de 224x224 píxeles.
    - Es posible reajustar el tamaño de las imágenes de entrada ya sea sin mantener o manteniendo la ratio de aspecto.

        - También requiere que las imágenes estén en el intervalo 0-255.
        - Si se quiere se puede realizar un incremento del conjunto de imágenes usando funciones de recorte aleatorio y giros en espejo o de orientación.
    - **VGG-16**
        - Incluido en keras.
        - 16 capas de profundidad.
        - Las capas más bajas extraen características de menor orden (colores, bordes) y las más altas características para determinar formas y objetos.
        - Las intermedias son las que definen el objeto que se está localizando.
    - **Inception-v3**.
        - Red muy profunda entrenada por Google.
        - Útil para la transferencia de aprendizaje.
- Usos de las redes convolucionales:
    - **Reconocimiento y localización de entidades en imágenes**
        - Se usa la métrica “Intersección sobre unión”
            - Area resultante de la unión entre el área etiquetada y predicha se le resta el área de la intersección entre el área etiquetada y predicha.
            - A más cercano a cero la resta, más idénticas son el área etiquetada y predicha.
    - **Detección múltiple de objetos**
        - Son redes convolucionales que no tienen capas densamente conectadas al final de la red. La capa convolucional debe tener:
            - Tantos filtros como neuronas existieran.
            - Los filtros con el mismo tamaño que la entrada esperada y con relleno “valid”.
        - La salida de estas redes será [tamaño del lote, 1, 1, número filtros].
        - Los pesos de la capa densa pueden ser transferidos a la capa convolucional.
        - Una de las arquitecturas más populares es YOLO.
    - **Segmentación semántica**.
        - Debido al uso de convoluciones, las imágenes se van reduciendo hasta que es casi indetectable las regiones.

- Para solucionar esto se usa el sobremuestreo: a partir de una imagen pequeña se genera una imagen de mayor resolución.
    - Primero, se va reduciendo el tamaño de la imagen hasta el tamaño deseado.
    - Se realiza un pequeño sobremuestreo para obtener la versión preliminar.
    - Al sobremuestreo se le añade mediante el salto de conexión el mapa de características que se tendría al principio
    - A la combinación de ambos se le aplicar un sobremuestreo mucho mayor para obtener el resultado final.



Procesamiento de texto

- Principales definiciones:
    - **Corpus**: conjunto de datos textuales en uno o varios lenguajes proveniente de libros, artículos, redes sociales o cualquier otro medio textual.
    - **Párrafo**: unidad más grande de texto soportada por una tarea de reconocimiento de lenguaje natural. Está formado por oraciones.
    - **Oración**: unidad con significado completo delimitada por signos de puntuación.
    - **Frases y palabras**: unidades de texto con significado limitado. Suelen ser los tokens en los que se dividen las oraciones.
    - **N-gramas**: Secuencia de n caracteres.
- Aplicaciones:
    - **Análisis de sentimientos**. Puntúa una oración en función de la opinión que expresa. Google Cloud proporciona una API para ello.
    - **Reconocimiento de entidades nombradas**. Identificación y clasificación de palabras o frases que tienen especial relevancia en el mundo real. API de Google Cloud.
    - **Enlazado de entidades**. Relacionado de las entidades detectadas. Microsoft Azure Text Analytics API.

    - **Traducción de texto**.
    - **Inferencia del lenguaje natural**. Clasifica la relación entre una premisa y una hipótesis.
    - **Etiquetado semántico del rol**. Determina qué rol tienen los diferentes elementos de una oración con respecto al verbo.
    - **Extracción de relaciones**.
    - **Conversión de lenguaje natural a consulta SQL**.
    - **Comprensión de texto.**
    - **Vinculación textual**. Determinar si existe implicación lógica correcta o si se trata de una contradicción.
    - **Resolución de correferencia**. Determina los pronombres que se refieren a la misma entidad.
    - **Búsqueda y recuperación de documentos**. Proporcionado por Bing API.
    - **Chatbots.** Como el caso de Amazon LEX.
    - **Conversión de texto a voz**. Proporcionado por Amazon Polly o por la herramienta text-to-speech de Google Cloud.
    - **Conversión de voz a texto**.
    - **Identificación de personas por voz**.
    - **Sistemas de diálogo hablado**. Como Siri o Alexa.
- Clasificación de textos:
    - Se aplica unas técnicas de preprocesado:
        - **Tokenización**. Conversión de las palabras en las unidades básicas. Normalmente se usan los signos de puntuación y los espacios.
        - **Lematización**. Eliminación de prefijos, sufijos y cualquier tipo de derivación para obtener el lema o la forma base de la palabra.
        - **Eliminación de palabras vacías**. Eliminación del texto de palabras que carecen de significado por sí mismas, como las preposiciones, adverbios, determinantes, pronombres, etc.
    - Tras el preprocesado se hace un análisis exploratorio del texto:
        - Conteo de palabras totales.
        - Conteo de frecuencias de palabras.
        - Obtención de la distribución de las palabras.
    - Uso de redes neuronales recurrentes:
        - La estructura básica es (secuencial):
            1. Capa encoder que transforme el texto a índices de tokens.
            2. Capa de incrustación para transformar estos índices de palabras en secuencias de vectores para poder realizar el entrenamiento.

3. Red neuronal recurrente.
    a. Procurar que sea bidireccional para que te tenga en cuenta el sentido de la oración tanto hacia un lado como hacia el otro.
    b. Se pueden indicar tantas capas como sea necesario:

```
model = tf.keras.Sequential([
    encoder,
    tf.keras.layers.Embedding(len(encoder.get_vocabulary()), 64, mask_zero=True),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64, return_sequences=True)),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(1)
])
```

4. Capa densa para agrupar la salida generada por la red neuronal recurrente.
5. Una última capa densa para determinar la salida de la clasificación.

- El entrenamiento del modelo se realizará usando una métrica relacionada con la clasificación.

- Aplicaciones de la clasificación de textos:
    - Etiquetado POS.
        - Consiste en asignar una categoría, como por ejemplo gramatical, a las palabras que quedan tras el preprocesado.
        - Se utiliza para:
            - Reconocimiento de entidades nombradas.
            - Análisis de sentimientos.
            - Resolución de preguntas.
            - Desambiguación del sentido de las palabras.
        - Se suele utilizar árboles o bosques para determinar la etiqueta de una palabra.
            - Junto con una palabra se puede añadir qué palabra le precede, cual le sucede, la distancia con respecto al inicio y final de la oración, si se trata de un dígito, etc.

        - Para determinar cómo de bueno es el modelo se puede usar cualquier tipo de métrica relacionada con problemas de clasificación.
- Análisis de sentimientos.
    - Se puede utilizar el modelo NaiveBayes.
        - Nos indica cuales son las palabras más determinantes para saber el sentido de la oración.
    - También se puede usar árboles y bosques.
    - Otro modelo es la bolsa de palabras.
        - Representación vectorial del texto.
        - Captura la presencia, frecuencia o ponderación de las palabras en el texto, sin capturar el orden.